CSA0307 - DATA STRUCTURES

Assignment – 1

Name: P. Bharath Kumar Reddy

Reg No: 192372050
Department: CSE - AI

1. What is Time Complexity? Explain with Examples (5 Marks)

Definition: Time complexity measures how the running time of an algorithm grows as the input size increases. It is commonly expressed using Big-O notation.

Examples of Time Complexity:

1. Constant Time — O(1): The operation takes roughly the same amount of time regardless of input size. Example: accessing arr[0] in an array.

2. Logarithmic Time — O(log n): The problem size is reduced by a constant factor at each step. Example: binary search in a sorted array.

3. Linear Time — O(n): The number of operations grows in proportion to input size. Example: scanning each element in a list using one loop.

4. Quadratic Time — O(n^2): The number of operations grows approximately as the square of input size. Example: two nested loops each running n times.

5. Exponential Time — O(2^n): The work can double for each extra input element. Example: generating every subset of a set.

Simple Python examples:

O(1)

```
arr = [10, 20, 30]
print(arr[0])
```

O(n)

```
for i in range(n):
    print(i)
```

$O(n^2)$

```
for i in range(n):
    for j in range(n):
        print(i, j)
```

Conclusion: Time complexity helps us compare algorithms and choose an efficient solution. In general, algorithms with lower time complexity perform better for large inputs.

2. Compare Iterative vs Recursive Factorial Computation (10 Marks)

Definition of Factorial

The factorial of a non-negative integer n is the product of all positive integers from 1 to n. It is written as n!. For example, 5! = 5 × 4 × 3 × 2 × 1 = 120, and 0! = 1.

A. Iterative Factorial

The iterative method uses a loop to multiply the numbers from 1 to n.

Python Program:

Program-2.py > ...

```python
def factorial_iterative(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

print("Factorial:", factorial_iterative(5))

```

Output: Factorial:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\padak\Downloads\CNS LAB PROGRAMS> & C:\Users\padak\AppData\Local\Programs\Python\Python312\python.exe "c:/Users
/padak/Downloads/CNS LAB PROGRAMS/Program-2.py"
Factorial: 120
PS C:\Users\padak\Downloads\CNS LAB PROGRAMS>
```

B. Recursive Factorial

The recursive method calls the same function with a smaller value until a base case is reached.

Formula: $n! = n \times (n - 1)!$ for $n > 1$. Base case: $0! = 1$ and $1! = 1$.

Python Program:

```python
def factorial_recursive(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial_recursive(n - 1)

print("Factorial:", factorial_recursive(5))
```

Output: Factorial:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\padak\Downloads\CNS LAB PROGRAMS> & C:\Users\padak\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/padak/Downloads/CNS LAB PROGRAMS/Program-2.py"
Factorial: 120
PS C:\Users\padak\Downloads\CNS LAB PROGRAMS>
```

Comparison Table

Feature	Iterative Method	Recursive Method
Technique	Uses a loop (for/while).	A function calls itself.
Termination	Stops when loop condition is false.	Stops when the base case is reached.
Time complexity	O(n).	O(n).
Auxiliary space	O(1), excluding input/output.	O(n) for recursive call stack.
Speed	Usually faster because there is no repeated function-call overhead.	May be slower due to function-call overhead.
Memory use	Uses little extra memory.	Stores function calls on the call stack.
Readability	Good for straightforward repetition.	Often closely matches the mathematical definition.

Risk	No recursion-depth limit issue.	Deep recursion may exceed the language recursion limit.

Conclusion

Both methods compute factorial in O(n) time. Iteration is generally more memory-efficient, while recursion can be elegant when a problem is naturally defined in terms of smaller subproblems.

3. Describe Recursion in Dynamic Programming Applications (5 Marks)

Definition

Recursion is a technique in which a function calls itself to solve smaller versions of a problem. Dynamic programming (DP) improves recursive solutions by storing the results of overlapping subproblems and reusing them instead of recalculating them.

Steps

1. Break the main problem into smaller subproblems.
2. Define base case(s) to stop recursive calls.
3. Use recursion to solve smaller subproblems.
4. Store results using memoization.
5. Reuse stored results to avoid repeated work and improve efficiency.

Example: Fibonacci Using Memoization

The Fibonacci sequence is defined as $F(n) = F(n-1) + F(n-2)$, with $F(0) = 0$ and $F(1) = 1$.

Python Program:

```python
def fib(n, memo=None):
    if memo is None:
        memo = {}
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fib(n - 1, memo) + fib(n - 2, memo)
    return memo[n]

print(fib(6))
```

Output: 8

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\padak\Downloads\CNS LAB PROGRAMS> & C:\Users\padak\AppData\Local\Programs\Python\Python312\python.exe "c:/Users
/padak/Downloads/CNS LAB PROGRAMS/Program-2.py"
8
PS C:\Users\padak\Downloads\CNS LAB PROGRAMS>
```

Applications of Recursion in Dynamic Programming

1. Fibonacci sequence: Calculates sequence values while storing previous results.

2. 0/1 Knapsack problem: Chooses items to maximize value without exceeding a weight limit.

3. Shortest path problems: Algorithms such as Floyd–Warshall find shortest paths between graph vertices.

4. Longest Common Subsequence (LCS): Finds the longest sequence shared by two strings; useful in text comparison.

5. Matrix-chain multiplication: Finds an efficient order for multiplying a chain of matrices.

Conclusion

Recursion expresses a problem as smaller subproblems, while dynamic programming stores and reuses their results. This can greatly reduce computation when subproblems overlap.